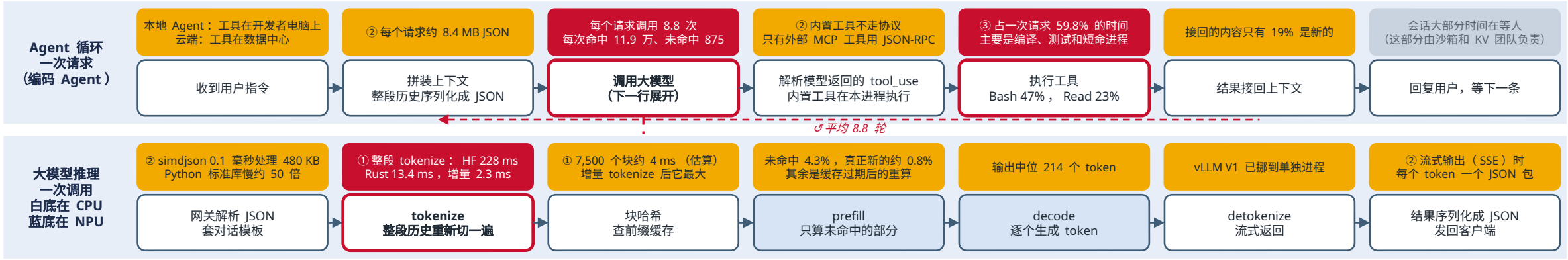
三个方向的轻重不一样：软件上先做 tokenizer，硬件上先看工具调用，JSON 只要把现成的快库用起来



① tokenizer
跑在推理服务器的 host CPU 或前端 CPU 池上

软件 P0 · 硬件 P1

业务特征： 编码 Agent 处理一个请求平均要调用模型 8.8 次。每次调用都把全部对话历史发过去，中位约 12 万个 token（480 KB 左右）。其中命中前缀缓存的中位 11.9 万个，未命中、要重新 prefill 的中位只有 875 个。按 token 加权，前缀缓存命中率 95.7%，真正新的内容约占 0.8%（TraceLab）。

负载特征： 推理前端每次都把整段上下文重新 tokenize，分三步：正则预分词把文本切成片段；片段拆成字节，在词表里查出初始 token ID；BPE 合并反复在合并表里找 rank 最小的相邻两个 token 合并，直到合并不了。运行时两张表都是哈希表（Qwen2.5 加 HuggingFace tokenizers，按数据结构估算）：合并表 15.1 万条，每条 16 字节（左 ID、右 ID、rank、新 ID 各 4 字节），约 4.5 MB；词表 15.2 万条，每条是“字符串 → 4 字节 ID”，槽位 32 字节再加一块存字符串的堆内存，约 13.5 MB，解码用的反向表也这么大。命中率趋近 0.99 时，tokenize 占首 token 延迟从 10% 涨到 64%（论文摘要的说法）。一段 450 KB 的上下文只用到合并表的 5% 到 7%（见第 4 页）。

当代软件发力点（950，现在就能做）： 做一个增量 tokenizer 库。算法换成线性 BPE；保存每个会话已切好的 token 序列，只重切末尾并校验边界，保证和参考实现逐个 token 一致。预分词用 950 的 256 位 SVE2（MATCH、HISTCNT）做字符分类。词表和合并表改用紧凑格式（最小完美哈希、按 rank 排序），整表 1 到 3 MB。块哈希和对话模板也做成增量。做成 HuggingFace tokenizers 的 drop-in 替换，接入 MindIE、vLLM 和 SGLang。目标是每 1,000 张加速卡所需的前端核从 114 个降到 1 个左右。

未来硬件发力点（960，软件做完后剩下的）： 软件做完以后，查表的热点只有 70 到 100 KB，950 每核 1 MB 的 L2 放得下，所以 L2 容量不是 tokenizer 的硬件诉求。剩下的是单核速度：tokenize 在首 token 的关键路径上，查表是依赖链上的随机访问，需要低的 L1 和 L2 命中时延，960 的高单核 SKU 适合做推理 host。tokenizer 和推理引擎同机会互相抢缓存，950 已支持 MPAM，可以先测隔离效果。不需要专门的 tokenizer 加速器。

② JSON
跑在 API 网关、Agent 框架和推理服务的 HTTP 层

软件 P2 · 硬件 P2

业务特征： 每次调用模型，Agent 都把整段对话历史（messages 数组）序列化成 JSON 发出去，API 网关再解析一遍，一个请求合计约 8.4 MB（估算）。内置工具不走 JSON-RPC，只有外部 MCP 工具才走。流式输出（SSE）时，每个 token 都包成一个小 JSON 发回客户端。

负载特征： 反序列化（以 simdjson 为例）分四步：① UTF-8 校验：逐字节查表，确认编码合法；② 建结构索引：逐字节分类出引号、反斜杠、括号、冒号、逗号和空白，用无进位乘法算出哪些字节在字符串里，再把结构字符的位置提取成下标数组；③ 建树：按下标走状态机，解析数字、字符串（还原转义）和 true、false、null；④ 转成 Python 或 Node 对象。序列化反过来：遍历对象，找出字符串里要转义的字符，把数字格式化成文本，二进制数据做 base64。① 和 ② 是向量操作，Neoverse V2/V3 上每核 5 到 7 GB/s，8.4 MB 约 2 毫秒；③ 是分支多的标量代码；④ 最慢，Python 标准库约 100 MB/s，慢在建对象。

当代软件发力点（950，现在就能做）： 不需要新算法，把 950 已有的指令用起来：① 和 ② 的字节分类用 TBL 查表和 SVE2 的 MATCH（一条指令比对 16 个字符）；引号区间用 PMULL 无进位乘法（SVE-AES 有 128 位版本）；位掩码的提取和压紧用 BitPerm 的 BEXT；base64 用 TBL 和 TBX。simdjson、simdutf 和 base64 库在 openEuler 里默认打开这些路径，并补上 256 位 SVE2 实现（今天只有 128 位 NEON）；orjson 这类 Python 绑定在 950 上实测后换成网关默认库。

未来硬件发力点（960，软件做完后剩下的）： 不做 JSON 专用硬件，960 补指令就够：SVE2.1 的 PMOV 把比较结果转成位图，TBLQ 按 128 位段查表；SVE2.2 的字节 COMPACT 把结构字符直接压成下标，FIRSTP 一条指令给出第一个命中的位置；CSSC 的 CTZ 一条指令数末尾有几个 0；LUTI4 用 4 位索引查小表，适合字节分类和 base64。架构基线定在 Armv9.6，这些都有。另外要把向量到标量的搬运做短，见第 3 页。

③ 工具调用
跑在云开发机、云端 Agent 和 RL rollout 的服务器上

软件 P1 · 硬件 P0

业务特征： Claude Code 的 14.2 万次工具调用里，Bash 占 47%，Read 占 23%，Edit 占 13%。工具执行占一次请求 59.8% 的时间，比模型生成还多。超过 1 分钟的调用只占 4%，却占工具时间的 85%，主要是编译、安装依赖和跑测试。每个请求约创建 5 到 7 个进程（估算）。数据来自 TraceLab，在开发者本地机器上测得。

负载特征： 编译器和测试框架代码量大、分支多，容易前端受限。每次工具调用都要起新进程，经过 fork、exec、动态链接、缺页和初始化，分支预测器、BTB 和 TLB 都是冷的；Node.js 的 libuv 在 Linux 上用 fork，父进程越大越慢。git 主要做 zlib 解压和大量 lstat，对象哈希默认是软件实现的 sha1dc，用不上 SHA 指令。Bash 里各命令的分布没有公开数据。

当代软件发力点（950，现在就能做）： 先用自家日志统计 Bash 里最常见的命令，再逐个改这几个库：① libuv：Node.js 创建子进程改用 posix_spawn（vfork），回合上游开发分支的改动，省掉 fork 大进程的页表复制；② glibc 和链接器：openEuler 默认链接选项加 RELR 紧凑重定位、去掉用不到的 .so，减少每次启动的动态链接开销；glibc 的 memchr、strchr 等字符串函数补 256 位 SVE2 版本；③ Rust 的 memchr 和 aho-corasick：补 256 位 SVE2 路径，ripgrep（Claude Code 的 Grep）直接受益；④ zlib-ng 和 libdeflate：git 对象和安装包的解压，校验和用 CRC32 指令和 PMULL。都通过 openEuler 软件包和 BoostKit 交付。

未来硬件发力点（960，软件做完后剩下的）： 编译和测试是加速库改不到的，只能靠核本身变快，这是三个方向里硬件收益最大的一块。960 要做：① 前端（P0）：加大 L3 到 L1-I 的取指带宽、BTB 容量和 ITLB，按编译、测试和短命进程的负载调优，不只看 SPEC；② 突发单核（P1）：按核睿频和快速 DVFS，高单核 SKU 给云开发机，256 核以上 SKU 给 RL rollout；③ 冷启动（P2，与沙箱团队合并提）：缩短缺页和系统调用的进出延迟，提高页清零带宽；④ 指令（P2）：补上 FEAT_MOPS（memcpy、memset 专用指令），架构基线 Armv9.6 自带 CTZ 和 FIRSTP，供 memchr 和 grep 用。

数据来源：TraceLab（华盛顿大学 2026，43 名开发者 8 个月、约 4,300 个会话的 Claude Code 和 Codex 本地使用日志；工具占比按调用次数算；59.8% 是工具执行占一次请求的时间，在开发者本地机器上测得）；TokTier（2026，采样 15.4 万次 Agent 调用，平均上下文 11.7 万 token，时间指占用一个核的时间）；AgentSysBench（港科大 2026，35,037 个生产会话）；Lemire 2026 年 9 月（simdjson 在 Graviton 4 和 5 上的单线程测试）；鲲鹏 950 的参数是团队内部口径（2026-09-23）。估算方法：JSON 8.4 MB = 480 KB × 编码和解析 2 次 × 8.8 次调用；查表大小按主流词表 10 万到 25 万条估；块哈希 4 毫秒 = 7,500 块 × 每块约 0.5 微秒。

960 的核与 SoC 该补什么：先补前端取指和单核速度，再做同机隔离和寄存器之间的搬运；不为 tokenizer、JSON 或正则单独做加速器

排序依据：只有硬件才能解决的排在前面（加速库改不到编译和测试）；三个方向都用得上的排在前面；能在 950 上马上测的排在前面。

要补的能力	主要为谁	为什么	950 现在	希望 960 做到	优先级	先在 950 上测什么
前端取指：L3 到 L1-I 的取指带宽、BTB 容量、ITLB	工具调用（编译、测试、短生命周期进程），也包括 Python 解释器	工具时间的 85% 花在编译、安装和测试上，加速库改不到；短命进程每次冷启动，容易前端受限；Google 全机群里前端停顿占 15% 到 30%（Kanev 2015）	取指带宽和 BTB 规模待内部确认	按开发者负载调优前端，不只看 SPEC	**P0（云开发机、云端 Agent）**	在 950 和 Graviton 4 上跑一组编译、测试、安装、git 和进程启动，用 top-down 分析看前端受限的比例
L2 和查表：每核 L2 容量、L2 和 L3 命中时延、在途缺失数（MLP）	动态链接、解释器和编译器；tokenizer 做了紧凑格式后不再需要	tokenizer 一次上下文的查表热点在紧凑格式下只有 70 到 100 KB（实测，见第 4 页），1 MB 的 L2 放得下；现有库的哈希表布局要占 1 MB 以上的缓存行（估算），这是软件问题	每核 1 MB，两个 SMT 线程共用	L2 容量等动态链接、解释器和编译器的实测再定；先保证 L2 和 L3 的命中时延	**P2**	tokenizer 紧凑格式前后的 L2 MPKI；动态链接和 Python 解释器的 L2 MPKI
突发单核性能和 SKU 分档	工具调用；tokenizer 在首 token 的关键路径上	Agent 循环是串行的；会话中位 80% 的时间在等人，忙时往往只有一两个核在跑（AgentSysBench）	—	按核睿频、快速 DVFS；96 核高单核 SKU 给云开发机和推理 host，256 核以上的 SKU 给 RL rollout	**P1**	固定频率和允许睿频两种设置下，同一组工具负载的完成时间
同机隔离：MPAM 分区和核间时延	tokenizer 和推理引擎放在同一台机器上时	CPU 欠配时首 token 会慢 1.47 到 7.11 倍；host 受干扰时 GPU kernel 间隙从 1 微秒拉长到 40 微秒（Blink）	已经支持 MPAM	SMT 两个线程之间也能分 L2；分区更细，支持 CMAX 容量上限；CSU、MBWU 监控计数齐全；片上网络支持优先级	**P1**	分别测不隔离、只绑核、加 L3 分区、再加带宽分区，看首 token 和 kernel 间隙
向量到标量的跨域搬运时延	三个方向里的 SIMD 文本处理	Arm 没有把谓词寄存器的位图直接搬进通用寄存器的指令；UMOV 在 Neoverse V2 上最多 10 个周期，在 Apple 的核上约 2 个周期（Lemire 2025）	团队实测偏慢	把这条路径做短，参考 Apple 的水平	**P1**	UMOV、CNTP、比较后跳转、STR P 后接 LDR 的转发时延
指令集基线	三个方向	950 缺 SVE2.1（TBLQ、PMOV）、SVE2.2（字节 COMPACT、FIRSTP）、CSSC（CTZ）和 MOPS（memcpy、memset 专用指令），都有替代写法，影响不大	SVE2，256 位，带 BitPerm、SVE-AES 等扩展	架构基线定在 Armv9.6 或以上，这些指令就都有了	**P2**	同一段扫描代码用三种写法比速度
会话的 token 序列放到 UB 内存池（设想）	tokenizer	增量 tokenize 需要找回上次的 token 序列，每个会话约 470 KB；放在共享内存池里，前端就不需要会话亲和	—	和 KV 存储团队一起定边界	**P2**	—
进程创建的内核路径：缺页处理、页清零、fork 页表复制	工具调用；RL rollout 更明显	用这些能力的是内核和沙箱，不是加速库；它在工具时间里占多少还没测过	—	不由我们直接提：我们在 950 上测出数据，交给内核和沙箱团队，和他们的沙箱创建诉求合在一起提	**协作**	bash -c true 和 python -c pass 的缺页次数、内核态时间占比
不做的事	—	tokenizer、JSON、正则都不做专用加速器：增量 tokenize 后每次只剩 2.3 毫秒，JSON 每次 0.1 毫秒，卸载到加速器反而要付来回开销	—	KAE 小数据块的方案先放一放	**—**	—

总的来说，960 的投入应该先放在通用核上：前端取指和单核速度，再加上 MPAM 同机隔离和向量到标量的跨域搬运。不需要为 tokenizer、JSON 或正则单独做加速器。每一项到底有多大，要看 950 上的五组测试（文档第 5.8 节）。

数据来源：TraceLab 2026（本地使用日志，工具时间占比）；AgentSysBench 2026（35,037 个生产会话，中位 80% 的时间空闲）；A CPU-Centric Perspective on Agentic AI 2025（48 核 Intel 服务器，并发从 1 到 128 时的能耗）；CPU 欠配论文 2026（多卡推理，首 token 慢 1.47 到 7.11 倍）；Blink 2026（host 受干扰时的 kernel 空隙）；Kanev 2015（Google 全机群的前端停顿）；Lemire 2025（UMOV 时延，Neoverse V2 的数字引自 Arm 的优化指南，Apple 的是估计值）；Arm 的特性寄存器说明和特性表（SVE 版本下限、CSSC）；鲲鹏 950 的参数是团队内部口径（2026-09-23）。

两个具体问题： 950 的向量指令够用，慢在向量到标量的跨域搬运； MPAM 只限制缓存分配，不影响进程之间通过缓存传数据

一、 950 的向量指令和向量到标量的跨域搬运

文本处理常用的指令	950 有没有	没有时怎么办
字符匹配 MATCH 、字符计数 HISTCNT 、两寄存器查表 TBL （SVE2）	有，256 位	—
按位抽取和放置 BEXT 、 BDEP （BitPerm，相当于 x86 的 PEXT 、 PDEP）	有	—
128 位无进位乘法 PMULLB 、 PMULLT （SVE-AES）， simdjson 和 CRC 会用到	有	—
按 128 位段查表 TBLQ （SVE2.1）	没有	用 LD1RQB 把 16 字节的表复制到每一段，再用普通 TBL
谓词和向量互转 PMOV （SVE2.1）	没有	用 STR P 把谓词直接存成位图；找第一个命中用 BRKB 加 CNTP
按字节 COMPACT 、直接给出第一个命中的 FIRSTP （SVE2.2）	没有	找第一个命中用 BRKB 加 CNTP； COMPACT 前先用 PUNPKLO/HI 把字节拆成 32 位
标量 CTZ，数末尾有几个 0 （CSSC）	没有	用 RBIT 加 CLZ 两条指令

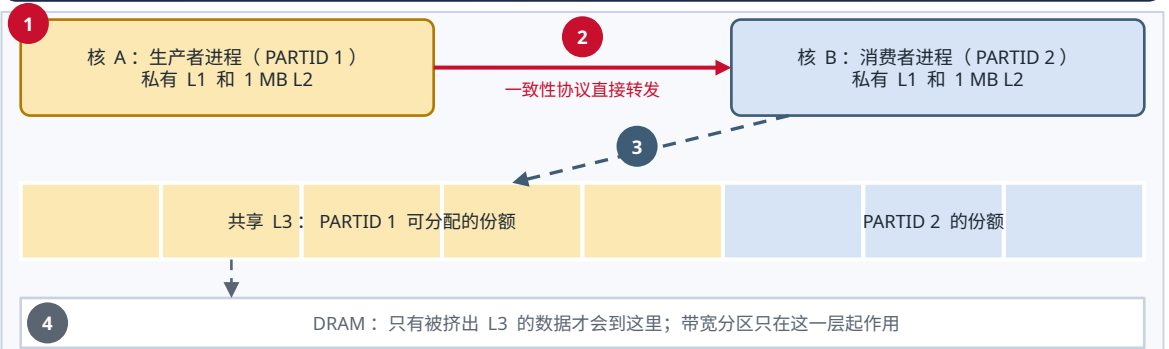
比较结果没法直接拿到通用寄存器里。 SVE 的比较结果放在谓词寄存器里，是一个位图。 Arm 到 SVE2.2 都没有指令能把这个位图直接搬进通用寄存器。能直接拿到的只有汇总值：命中个数（CNTP）、第一个命中的位置（FIRSTP， SVE2.2 才有）、有没有命中（比较指令会直接设置 NZCV 条件标志）。 SVE2.1 的 PMOV 也只能搬到向量寄存器，最后那一步 UMOV 还是省不掉。

这一步慢是微架构决定的。 向量寄存器堆和整数寄存器堆在物理上是分开的，跨域搬运要经过专门的传输端口和旁路网络，向量越宽通常离得越远。 Lemire 2025 的测量表里， UMOV 在 Neoverse V2 上最多 10 个周期，在 Apple 的核上约 2 个周期。团队在 950 上也看到这一步偏慢。

软件上别让它卡在循环的依赖链上。 能用条件标志判断的先判断，真有命中再算位置；每 64 字节只搬一次 64 位掩码；或者用 STR P 把位图批量写进 L1 缓冲区，等向量阶段跑完再由标量代码顺序读，比的是吞吐而不是时延。 simdjson 的两阶段设计（先建结构索引，再建树）就是这样， tokenizer 的预分词也可以照做。

对 960 的希望。 缩短三条跨域路径：向量或谓词寄存器到整数寄存器、 NZCV 标志到分支单元、 STR P 到 LDR 的 store-to-load 转发，参考 Apple 约 2 个周期的水平。架构基线定在 Armv9.6 或以上，缺的指令就都有了。“谓词位图直接进通用寄存器”的指令 Arm 架构里没有， A 系列也不允许自定义指令，只能通过 Arm 的架构合作渠道提。

二、 MPAM 隔离和进程之间的通信



① A 写共享缓冲区，缓存行先留在 A 的私有 L2。② B 读的时候，一致性协议把这一行从 A 的 L2 直接转给 B。③ 也可能在 L3 命中，即使这一行在 PARTID 1 的份额里。④ 只有被挤出 L3 才去 DRAM。

MPAM 管的是缓存怎么分，不管谁能读。 MPAM 给每个线程的访存请求打上分区号（PARTID）。缓存按分区号限制新数据能分配到哪些份额（CPOR 位图）、最多占多少（CMAX）。查找不受限制，一个分区的请求可以命中别的分区分配进来的缓存行，缓存一致性协议也照常工作。所以 MPAM 是性能隔离，不是安全隔离。

同一台机器上的进程通信本来就走缓存。 共享内存只是软件上的说法，数据实际是通过一致性协议在两个核的缓存之间传的。管道、 Unix 域套接字（ZeroMQ 的 ipc 方式）和本机回环 TCP 也一样，只是多了系统调用和内核拷贝。只有跨机器才需要网络或 UB。

隔离之后要注意几件事。 共享缓冲区的缓存行记在先把它拉进缓存的那个进程的份额里，份额太小可能在被读之前就被挤出 L3，所以环形缓冲区要做小、循环复用。带宽分区只在数据落到 DRAM 时才起作用。时延取决于两个核离得多远，频繁通信的进程要绑在同一个 cluster； tokenizer 不要和推理引擎的 host 线程放在同一个物理核的两个 SMT 线程上。消费者可以用 LDAXR 加 WFE 等在标志位上，生产者一写就被唤醒。

对推理 host 来说，先测隔离。 每一步的进程间消息只有几 KB 到几百 KB，而每一步 decode 要几毫秒，通信本身占不到 1% （估算）。真正的问题是互相干扰。

对 960 的希望。 SMT 两个线程之间也能分 L2； 分区更细， CSU 和 MBWU 监控计数齐全；片上网络能给关键线程更高优先级。还可以支持 Armv9.6 的生产者 - 消费者放置提示（FEAT_PCDPHINT 的 STSHH 和 PRFM IR），让写完的数据提前推到消费者附近； CHI 协议里现成的 stash 操作如果开放给 CPU，就能做到核到核直送。

数据来源： Lemire 等 2025 （Scanning HTML at Tens of Gigabytes per Second，表 2； Neoverse V2 的数字引自 Arm 的软件优化指南， Apple 的是估计值）； Arm 的特性寄存器说明、 Armv8.9 和 Armv9.6 的特性说明，以及 LLVM 、 QEMU 、 binutils 的相关补丁（SVE2.1 、 SVE2.2 、 SVE2.3 、 CSSC 、生产者 - 消费者放置提示）； Arm 专利 US11237985 和 SemiEngineering 介绍的 gem5 MPAM 实现（分配和查找的区别）； CPU 欠配论文 2026 、 Blink 2026；鲲鹏 950 的参数是团队内部口径（2026-09-23）。估算方法：通信占比按每一步的消息量和每一步 decode 的时间比较得出。

tokenizer 的词表和合并表：原始文件 1.4 到 9 MB、加载后 24 到 77 MB；改成紧凑格式后整表 1 到 3 MB，一次上下文真正用到的热点只有 70 到 100 KB

一、原始有多大，紧凑格式能压到多小

模型	词表条数	合并规则条数	原始文件	现有库加载后内存	紧凑格式：合并表	紧凑格式：词表
GPT-2	50,257	50,000	1.36 MB	24 MB （HF）	**0.42 MB**	**0.47 MB**
GPT-4 （cl100k）	100,256	约 10 万	1.68 MB	42 MB （tiktoken）	**0.74 MB**	**0.94 MB**
DeepSeek-V3	128,815	127,741	7.85 MB	59 MB （HF）	**1.20 MB**	**1.22 MB**
Llama 3	128,256	280,147	9.09 MB	70 MB （HF）	**2.63 MB**	**1.22 MB**
Qwen2.5	151,665	151,387	7.03 MB	59 MB （HF）	**1.42 MB**	**1.43 MB**
GPT-4o （o200k）	199,998	约 20 万	3.61 MB	77 MB （tiktoken）	**1.47 MB**	**2.00 MB**

原始文件：HuggingFace 的 tokenizer.json 或 OpenAI 的 .tiktoken 文件。加载后内存：用 HuggingFace tokenizers 0.23 或 tiktoken 0.14 加载后进程常驻内存的增量，含正则编译和解析时未归还的内存，是上界。tiktoken 没有显式合并表，合并规则数按“词表减 256”算。Llama 3 的 tokenizer.json 里有 28 万条合并规则，比词表还多。

二、怎么压到这个大小，现有库为什么这么大

token ID 只需要 17 到 18 位。词表 13 万到 20 万条，2 的 18 次方是 26 万，够用。

合并表每条只存三到四个 ID。存左 token 和右 token，用来精确核对是不是这一对；再存合并的优先级（rank）。tiktoken 格式里 rank 就是新 token 的 ID，每条 $3 \times 18 = 54$ 位，按 7 字节存；HuggingFace 格式还要多存一个新 ID，按 9 字节存。

用最小完美哈希（MPH）定位。PTHash、CHD 这类算法给 n 个键分配 0 到 n−1 的唯一位置，每个键额外只要 2 到 3 位，没有空槽，也不用指针。20 万条按 7 字节一条约 1.4 MB，加上哈希参数约 1.5 MB（之前按 8 字节一条估的是 1.6 MB）。

按 rank 排序存放。BPE 的合并规则本来就是按语料里出现的频率学出来的，rank 越小越常用。按 rank 排好，常用的条目自然挤在表的前面。

词表只在解码时用。把所有 token 的字节拼成一整块，再加一个 3 字节的偏移数组。token 平均 6.4 到 7 字节，20 万条约 2 MB。

现有库为什么大。HuggingFace tokenizers 用 Rust 哈希表存“(左，右)→(rank，新 ID)”，每条 16 字节，再加控制字节和空槽；词表是“字符串 → ID”和“ID → 字符串”两张哈希表，每个字符串单独分配一块内存。tiktoken 用“字节串 → rank”的哈希表，每个键也是单独分配的，Python 侧还另存一份字典。

三、一次上下文真正用到多少（实测）

5%–7%

一段 450 KB 的上下文
用到的合并规则占全表

70–100 KB

这些条目按紧凑格式
只占这么大

GPT-4o （o200k）：切出 15.4 万个 token，用到 10,773 条合并规则，占全表 5.4%，按 7 字节一条是 74 KB。99% 的查找落在其中 8,757 条里（60 KB）；约 70% 的查找落在 rank 小于 1 万的条目里。

Qwen2.5：切出 16.1 万个 token，用到 11,342 条，占全表 7.5%，按 9 字节一条是 100 KB；99% 的查找落在 8,467 条里（74 KB）。

对 950 意味着什么。整张表压紧后 1 到 3 MB，比每核 1 MB 的 L2 大；但一次上下文的热点只有 70 到 100 KB，放得下。现有库的哈希表把每个热点条目散在各自的缓存行上，估算要占 1 MB 以上。所以查表的问题主要靠软件解决。

四、已经有人这样做过

GitHub 的 bpe 库（Rust，rust-gems）：所有 token 的字节拼成一整块，再用一个偏移数组定位；查找键用 32 位哈希值代替字节串，构建时选一个乘数保证不冲突；合并对另有一张（左，右）→ 新 token 的表；最长匹配用双数组 Aho-Corasick 自动机。线性 BPE 也出自这个库。

Microsoft BlingFire：把分词算法和词表编译成有限状态机，存成二进制模型文件，提供 GPT-2 和 RoBERTa 的 byte-BPE 模型（gpt2.bin、roberta.bin）。

SentencePiece：Unigram 模型的词表用双数组 trie（Darts-clone）存，紧凑、查找快。

Google Fast WordPiece （EMNLP 2021）：用 trie 加失败链接做线性时间切分，模型序列化成 flatbuffer。它是 WordPiece，不是 BPE，但存储思路相同。

HuggingFace tokenizers 主干（tk-encode，1.0.0-dev，未发布）：token ID 按 rank 重排，最热的合并放进 512×512 的稠密网格，其余的对用 PtrHash 完美哈希；每条存 16 字节，键是（左 << 32 | 右），值把 rank 和新 token ID 打包进一个 u64。

数据来源与口径：2026-09-23 在本机实测。词表文件取自 HuggingFace （openai-community/gpt2、meta-llama/Meta-Llama-3-8B 的 NousResearch 镜像、Qwen/Qwen2.5-7B-Instruct、deepseek-ai/DeepSeek-V3）和 OpenAI 的 tiktoken 公开文件（cl100k_base、o200k_base）。紧凑格式大小按上面的编码方法计算，不是实现后的实测。热点实测用 449 KB 的混合文本（JavaScript 代码 280 KB、中英文 Markdown 141 KB、JSON 28 KB），按原库的 BPE 算法逐步重放、统计查到的合并规则，输出与 tiktoken 和 HuggingFace tokenizers 逐个 token 一致。“1 MB 以上”是按哈希表布局估算的。案例来自各项目的源码和文档。